

EDA — Helmet Dataset (OSF Lite)

Dataset: <https://www.kaggle.com/datasets/kronomy/helmet-dataset-by-osf-lite>

This is an **image classification** dataset (not tabular/CSV), so this notebook does image-focused EDA:

- downloads the dataset directly on Colab using `kagglehub`
- auto-detects the class folders
- counts images per class
- checks for corrupted/unreadable images
- looks at image size / aspect ratio distribution
- shows sample images per class

1. Install & import dependencies

```
[1] ✓ 18
!pip install -q kagglehub

import os
import random
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image, UnidentifiedImageError
import kagglehub
```

2. Download the dataset (Kaggle API via kagglehub)

`kagglehub` needs your Kaggle credentials. On Colab, upload your `kaggle.json` (Account -> Create New API Token on kaggle.com), or run `kagglehub.login()` to authenticate interactively.

```
[1] ✓ 18
# Option A: Interactive login (uncomment if you don't have kaggle.json set up)
# kagglehub.login()

# Option B: upload kaggle.json (Colab file upload), then place it correctly
try:
    from google.colab import files
    kaggle_json_path = os.path.expanduser("~/kaggle/kaggle.json")
    if not os.path.exists(kaggle_json_path):
        print("Upload your kaggle.json file (skip this if already configured).")
        uploaded = files.upload()
        if "kaggle.json" in uploaded:
            os.makedirs(os.path.expanduser("~/kaggle"), exist_ok=True)
            with open(kaggle_json_path, "wb") as f:
                f.write(uploaded["kaggle.json"])
            os.chmod(kaggle_json_path, 0o600)
except ImportError:
    # Not running on Colab
    pass

Upload your kaggle.json file (skip this if already configured):
kaggle.json
kaggle.json(application/json) - 72 bytes, last modified: 25/08/2026 - 100% done
Saving kaggle.json to kaggle.json

# Download dataset (cached after first run)
dataset_path = kagglehub.dataset_download("kronomy/helmet-dataset-by-osf-lite")
print("Dataset downloaded to:", dataset_path)
```

```
[1] ✓ 18
Downloading to /root/.cache/kagglehub/datasets/kronomy/helmet-dataset-by-osf-lite/1.archive...
100%[██████████] 8.82G/8.82G [07:12:00:00, 21.9MB/s]Extracting files...

Dataset downloaded to: /root/.cache/kagglehub/datasets/kronomy/helmet-dataset-by-osf-lite/versions/1
```

3. Explore the folder structure

```
[1] ✓ 18
for root, dirs, files in os.walk(dataset_path):
    level = root.replace(dataset_path, "").count(os.sep)
    indent = " " * 2 * level
    print(f'{indent}(os.path.basename(root))')
    if level < 2:
        sub_indent = " " * 2 * (level + 1)
        for f in files_[:5]:
            print(f'{sub_indent}(f)')
        if len(files_) > 5:
            print(f'{sub_indent}... ({len(files_)} files total)')

-----
Mandalay_1_2/
Mandalay_1_94/
Mandalay_1_46/
Mandalay_1_129/
Mandalay_1_50/
Bago_urban_61/
Mandalay_1_80/
Mandalay_1_40/
Mandalay_1_11/
Mandalay_1_38/
Mandalay_1_77/
Mandalay_1_23/
Mandalay_1_42/
Mandalay_1_63/
Mandalay_1_34/
Mandalay_1_15/
Mandalay_1_26/
Mandalay_1_90/
Mandalay_1_10/
Mandalay_1_183/
Mandalay_1_99/
Mandalay_1_75/
Mandalay_1_93/
Mandalay_1_84/
Mandalay_1_25/
Mandalay_1_18/
Mandalay_1_54/
Mandalay_1_124/
Mandalay_1_59/
Mandalay_1_7/
Mandalay_1_73/
Mandalay_1_85/
Mandalay_1_86/
Mandalay_1_6/
Mandalay_1_132/
Mandalay_1_16/
Mandalay_1_66/
Mandalay_1_51/
Mandalay_1_32/
Bago_urban_56/
Mandalay_1_130/
Mandalay_1_47/
Mandalay_1_92/
Mandalay_1_53/
Mandalay_1_12/
Mandalay_1_24/
Mandalay_1_20/
Bago_urban_45/
Mandalay_1_4/
Bago_urban_62/
Mandalay_1_83/
Mandalay_1_22/
Mandalay_1_58/
Bago_urban_42/
Bago_urban_48/
Bago_urban_50/
Bago_urban_46/
Mandalay_1_17/
```

4. Auto-detect class folders and count images

Assumes a standard image-classification layout: `dataset_root/<class_name>/<image files>` (searches subfolders if images aren't directly under the root).

```
[1] ✓ 18
IMG_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".gif", ".webp")

def find_class_dirs(root):
    """Find directories that directly contain image files - treated as class folders."""
    class_dirs = {}
    for dirpath, dirnames, filenames in os.walk(root):
        images = [f for f in filenames if Path(f).suffix.lower() in IMG_EXTS]
        if images:
            class_name = os.path.basename(dirpath)
            class_dirs.setdefault(class_name, []).extend(
                [os.path.join(dirpath, f) for f in images]
            )
    return class_dirs

class_dirs = find_class_dirs(dataset_path)

print("Detected classes:")
for cls, imgs in class_dirs.items():
    print(f"  {cls}: {len(imgs)} images")

-----
Mandalay_1_2: 100 images
Mandalay_1_94: 100 images
Mandalay_1_46: 100 images
Mandalay_1_129: 100 images
Mandalay_1_50: 100 images
Bago_urban_61: 100 images
Mandalay_1_80: 100 images
Mandalay_1_40: 100 images
Mandalay_1_11: 100 images
Mandalay_1_38: 100 images
Mandalay_1_77: 100 images
Mandalay_1_23: 100 images
Mandalay_1_42: 100 images
Mandalay_1_63: 100 images
Mandalay_1_34: 100 images
Mandalay_1_15: 100 images
Mandalay_1_26: 100 images
Mandalay_1_90: 100 images
Mandalay_1_10: 100 images
Mandalay_1_183: 100 images
Mandalay_1_99: 100 images
Mandalay_1_75: 100 images
Mandalay_1_93: 100 images
Mandalay_1_84: 100 images
Mandalay_1_25: 100 images
Mandalay_1_18: 100 images
Mandalay_1_54: 100 images
Mandalay_1_124: 100 images
Mandalay_1_59: 100 images
Mandalay_1_7: 100 images
Mandalay_1_73: 100 images
Mandalay_1_85: 100 images
Mandalay_1_86: 100 images
Mandalay_1_6: 100 images
Mandalay_1_132: 100 images
Mandalay_1_16: 100 images
Mandalay_1_66: 100 images
Mandalay_1_51: 100 images
Mandalay_1_32: 100 images
Bago_urban_56: 100 images
Mandalay_1_130: 100 images
Mandalay_1_47: 100 images
Mandalay_1_92: 100 images
Mandalay_1_53: 100 images
Mandalay_1_12: 100 images
Mandalay_1_24: 100 images
Mandalay_1_20: 100 images
Bago_urban_45: 100 images
Mandalay_1_4: 100 images
Bago_urban_62: 100 images
Mandalay_1_83: 100 images
Mandalay_1_22: 100 images
Mandalay_1_58: 100 images
Bago_urban_42: 100 images
Bago_urban_48: 100 images
Bago_urban_50: 100 images
Bago_urban_46: 100 images
Mandalay_1_17: 100 images

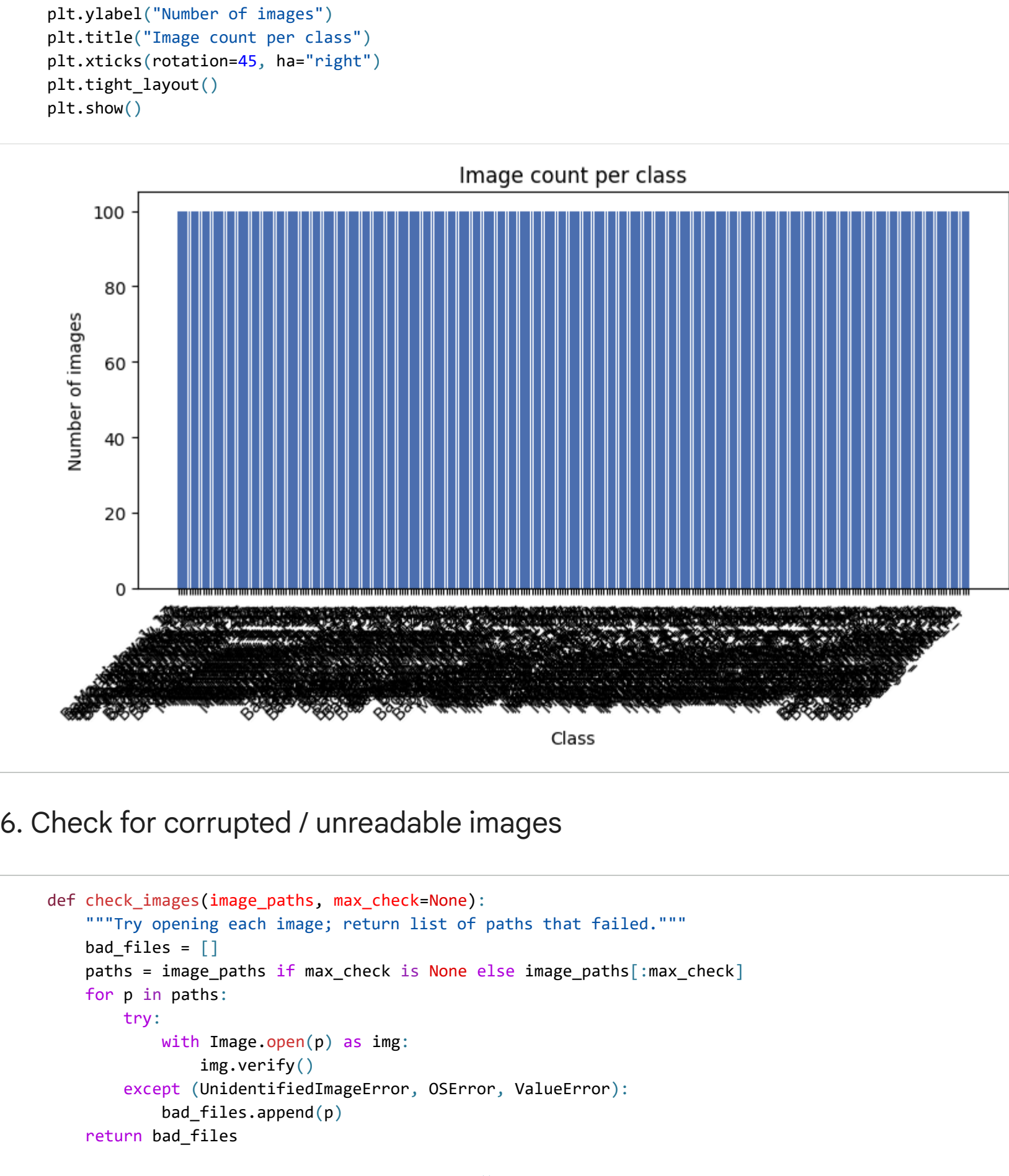
class_counts = {cls: len(imgs) for cls, imgs in class_dirs.items()}
counts_df = pd.DataFrame(list(class_counts.items()), columns=["class", "num_images"]).sort_values("num_images", ascending=False)
counts_df

-----
class  num_images
286  Mandalay_1_17  100
0    Bago_highway_10  100
1    Bago_highway_17  100
2    Bago_rural_46  100
3    Bago_highway_28  100
...  ...  ...
34   Bago_urban_12  100
35   Bago_highway_11  100
36   Bago_urban_24  100
37   Bago_rural_43  100
38   Bago_urban_4  100

287 rows × 2 columns

Next steps: Generate code with counts\_df New interactive sheet
```

5. Class distribution plot



6. Check for corrupted / unreadable images

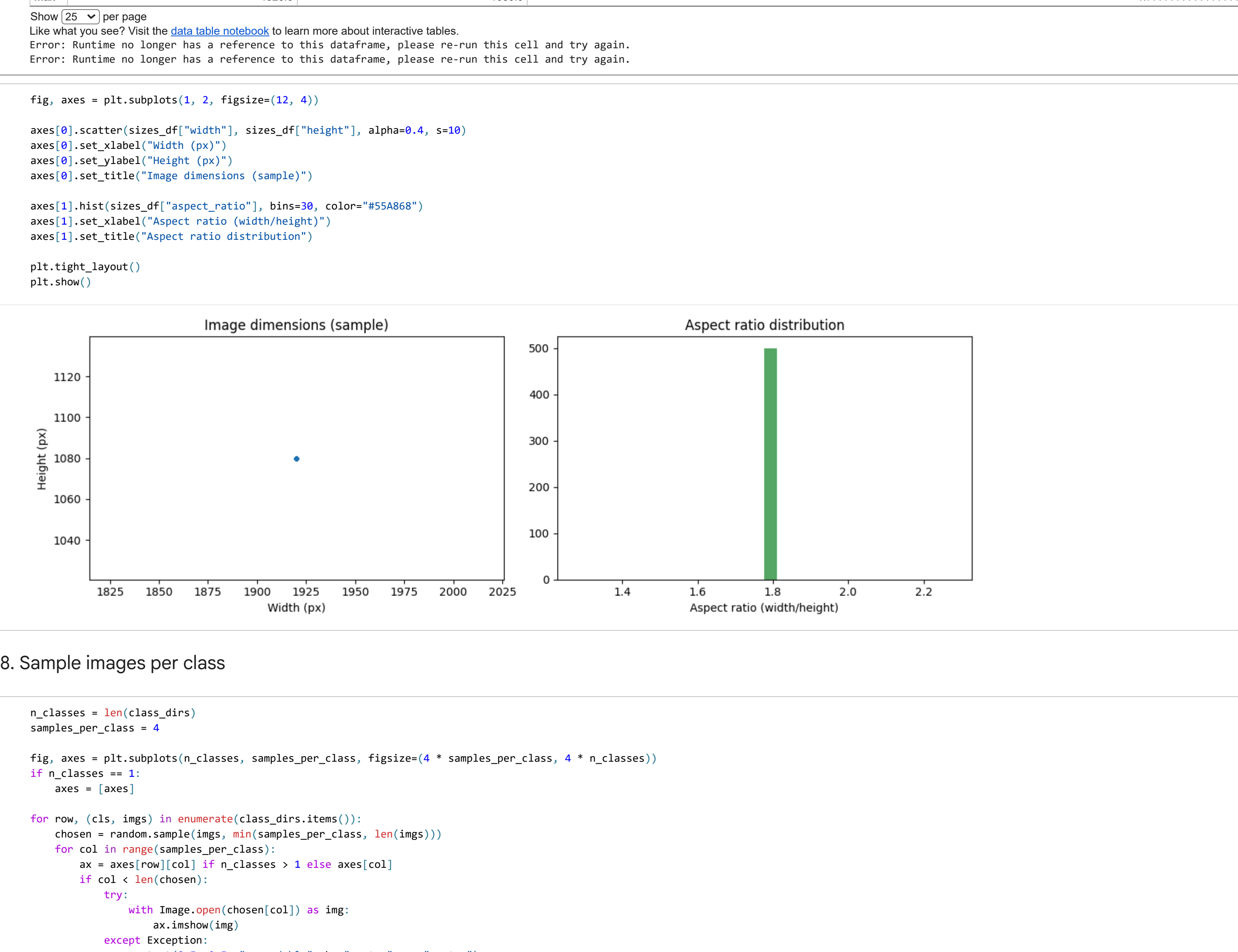
```
[1] ✓ 26
def check_images(image_paths, max_check=None):
    """Try opening each image; return list of paths that failed."""
    bad_files = []
    paths = image_paths if max_check is None else image_paths[:max_check]
    for p in paths:
        try:
            with Image.open(p) as img:
                img.verify()
        except UnidentifiedImageError, OSError, ValueError:
            bad_files.append(p)
    return bad_files

all_images = [p for imgs in class_dirs.values() for p in imgs]
print(f"Total images found: {len(all_images)}")

# Set max_check=None to check all images (can be slow for large datasets)
bad_files = check_images(all_images, max_check=None)
print(f"Corrupted/unreadable images: {len(bad_files)}")
if bad_files:
    for f in bad_files[:10]:
        print(f"- {f}")

Total images found: 28700
Corrupted/unreadable images: 0
```

7. Image size & aspect ratio distribution



8. Sample images per class

```
[11] ✓ 18
n_classes = len(class_dirs)
samples_per_class = 4

fig, axes = plt.subplots(n_classes, samples_per_class, figsize=(4 * samples_per_class, 4 * n_classes))
if n_classes == 1:
    axes = [axes]

for row, (cls, imgs) in enumerate(class_dirs.items()):
    chosen = random.sample(imgs, min(samples_per_class, len(imgs)))
    for col in range(samples_per_class):
        ax = axes[row][col] if n_classes > 1 else axes[col]
        if col < len(chosen):
            try:
                with Image.open(chosen[col]) as img:
                    ax.imshow(img)
            except Exception:
                ax.text(0.5, 0.5, "unreadable", ha="center", va="center")
        ax.axis("off")
        if col == 0:
            ax.set_ylabel(cls, fontsize=12)

plt.tight_layout()
plt.show()
```

9. Summary

```
[12] ✓ 18
print("EDA summary")
print("-" * 40)
print(f"Total classes: {n_classes}")
print(f"Total images: {len(all_images)}")
print(f"Corrupted images: {len(bad_files)}")
print(f"Sampled image size stats (px):")
print(sizes_df[["width", "height"]].describe().loc[["mean", "min", "max"]])

EDA summary
-----
Total classes: 287
Total images: 28700
Corrupted images: 0
Sampled image size stats (px):
      width  height
mean  1920.0  1080.0
min    1920.0  1080.0
max    1920.0  1080.0
```